DOCUMENTACIÓN TÉCNICA OFICIAL · VERSIÓN 1.0.0

Plataforma de Trueque Estudiantil

Manual Técnico y Memoria de Arquitectura del Backend

El presente documento constituye la memoria técnica del componente **backend** de la Plataforma de Trueque Estudiantil: un sistema web que permite a estudiantes universitarios intercambiar libros de texto, apuntes, calculadoras y material académico sin mediar dinero. Se documenta la arquitectura de software adoptada (Hexagonal / Puertos y Adaptadores), el modelo de datos y su persistencia asíncrona, el módulo de autenticación y seguridad criptográfica, el catálogo completo de endpoints de la API REST, la estrategia de pruebas automatizadas, el procedimiento de despliegue y el estado real de implementación frente a los requerimientos levantados.

El sistema está construido sobre **FastAPI**, **SQLAlchemy 2.0 Async**, **PostgreSQL** y **Pydantic v2**, con control de acceso basado en roles (RBAC), hashing de contraseñas con bcrypt, emisión de tokens JWT conformes a RFC 7519 y refresh tokens almacenados en cookies HttpOnly.

Ficha Técnica del Proyecto

Nombre del sistema	Plataforma de Trueque Estudiantil
Componente documentado	Backend RESTful API — Módulo de Usuarios y Autenticación
Tecnologías centrales	FastAPI · SQLAlchemy 2.0 Async · PostgreSQL 15 · Pydantic v2 · PyJWT · Bcrypt
Patrón arquitectónico	Arquitectura Hexagonal (Puertos y Adaptadores / Clean Architecture)
Metodología de trabajo	Ágil — Scrumban (sprints planificados + flujo continuo en Trello)
Control de versiones	Git / GitHub · `pre-commit` con Black y Ruff
Pruebas	Pytest + HTTPX ASGITransport + Respx — 8 de 8 pruebas aprobadas
Persistencia	PostgreSQL 15 (producción / Docker) · SQLite asíncrono (pruebas aisladas)
Contenerización	Docker + Docker Compose (API + base de datos con volumen persistente)
Estado de la entrega	Versión 1.0.0 — Backend funcional y documentado
Fecha	Septiembre de 2026

Tabla de Contenido

1. Introducción, contexto y alcance
    - 1.1 Planteamiento del problema
    - 1.2 ¿Qué es y para qué sirve el backend?
    - 1.3 Objetivos del sistema
    - 1.4 Alcance de esta entrega
    - 1.5 Matriz de requerimientos funcionales y no funcionales
    - 1.6 Estado real de implementación
2. Metodología de desarrollo y stack tecnológico
    - 2.1 Metodología Scrumban
    - 2.2 Stack tecnológico y justificación de cada elección
    - 2.3 Calidad de código: Black, Ruff y pre-commit
3. Arquitectura Hexagonal (Puertos y Adaptadores)
    - 3.1 Motivación de la decisión arquitectónica
    - 3.2 Diagrama de capas
    - 3.3 Responsabilidad de cada capa
    - 3.4 Estructura de carpetas del proyecto
    - 3.5 Inyección de dependencias
    - 3.6 Trazado de una petición de extremo a extremo
4. Base de datos y persistencia
    - 4.1 Configuración visible de la conexión
    - 4.2 Motor asíncrono y sesión por petición
    - 4.3 Diccionario de datos: tabla `usuarios`
    - 4.4 Modelo entidad-relación proyectado
    - 4.5 Migraciones con Alembic
5. Módulo de autenticación y seguridad
    - 5.1 Hashing de contraseñas con bcrypt
    - 5.2 Tokens JWT y el identificador único JTI
    - 5.3 Cookies HttpOnly para el refresh token
    - 5.4 Flujo completo de autenticación
    - 5.5 Control de acceso basado en roles (RBAC)
    - 5.6 Inicio de sesión federado con Google OAuth2
    - 5.7 Riesgos identificados y recomendaciones
6. Contratos de datos: esquemas Pydantic v2
7. Catálogo completo de endpoints REST
    - 7.1 Matriz de rutas
    - 7.2 Ejemplos de petición y respuesta
    - 7.3 Manejo estándar de errores
8. Pruebas automatizadas y aseguramiento de calidad
9. Despliegue y puesta en marcha
    - 9.1 Requisitos del entorno
    - 9.2 Despliegue local paso a paso
    - 9.3 Despliegue con Docker Compose
    - 9.4 Variables de entorno
10. Limitaciones actuales y trabajo futuro
11. Conclusiones
12. Glosario de términos técnicos

1. Introducción, contexto y alcance

1.1 Planteamiento del problema

Cada inicio de semestre, un estudiante universitario enfrenta un gasto significativo en material académico: libros de texto que usará durante cuatro meses, calculadoras científicas o graficadoras exigidas por una sola asignatura, batas de laboratorio, guías y apuntes. Simultáneamente, estudiantes de semestres superiores conservan ese mismo material sin uso, ocupando espacio y perdiendo valor.

El canal actual para resolver este desencuentro son los grupos informales de WhatsApp y Facebook de cada facultad, donde las publicaciones se pierden en el flujo de mensajes, no existe forma de buscar por materia o título, y no hay ningún mecanismo de confianza que respalde al desconocido con el que se pacta el intercambio. El resultado es un mercado con información dispersa, alta fricción de búsqueda y riesgo reputacional no gestionado.

La **Plataforma de Trueque Estudiantil** centraliza esa oferta y esa demanda en un sistema estructurado, buscable y con reputación verificable, eliminando además el dinero de la ecuación: el valor se intercambia en especie, bien por bien.

1.2 ¿Qué es y para qué sirve el backend?

El backend es el motor computacional y lógico central del sistema. Es la pieza que el usuario nunca ve pero de la que depende absolutamente todo lo que sí ve. Sus responsabilidades son:

- **Custodiar la información crítica:** cuentas, credenciales, perfiles académicos y — en fases posteriores — publicaciones, trueques y valoraciones.
- **Ejecutar las reglas de negocio:** quién puede publicar, quién puede aceptar un trueque, cuándo una negociación cambia de estado, qué constituye un perfil válido.
- **Autenticar y autorizar:** verificar que quien dice ser un estudiante lo sea, y que cada acción esté permitida para su rol.
- **Validar la integridad de cada solicitud:** ningún dato llega a la base de datos sin haber superado un contrato estricto de tipos y formato definido con Pydantic v2.
- **Persistir de forma asíncrona:** escribir y leer en PostgreSQL sin bloquear el hilo de ejecución, permitiendo atender múltiples estudiantes concurrentemente.
- **Exponer una API REST:** un contrato público y versionable que alimenta indistintamente a un cliente web o a una futura aplicación móvil.

1.3 Objetivos del sistema

- **Intercambio solidario sin dinero.** Facilitar el trueque de libros de texto, calculadoras científicas, apuntes y batas de laboratorio entre compañeros de universidad.
- **Eficiencia en la búsqueda.** Reducir a segundos el tiempo que un estudiante invierte localizando el material que necesita para su semestre.
- **Economía circular y sostenibilidad.** Promover la reutilización de recursos educativos dentro del campus, extendiendo la vida útil de cada libro.
- **Entorno de confianza.** Respaldar cada intercambio con un sistema de valoraciones de 1 a 5 estrellas y perfiles asociados a un correo institucional verificable.
- **Base técnica mantenible.** Construir sobre una arquitectura desacoplada que permita al equipo incorporar los módulos de publicaciones, matching y trueques sin reescribir el núcleo.

1.4 Alcance de esta entrega

Esta versión 1.0.0 documenta y entrega el **módulo de usuarios y autenticación**, que constituye el cimiento obligatorio de todos los demás módulos: sin identidad verificada no hay publicación, ni trueque, ni reputación atribuible. Los módulos de publicaciones, motor de matching, gestión del trueque y valoraciones están diseñados y modelados en este documento, pero su implementación corresponde a las siguientes iteraciones. La sección 1.6 detalla con precisión qué está construido y qué no.

1.5 Matriz de requerimientos funcionales y no funcionales

ID	Requerimiento	Descripción técnica
RF-01	Registro e inicio de sesión	Creación de cuentas con validación de formato de correo, hashing de contraseña y emisión de tokens JWT con cookie HttpOnly.
RF-02	Gestión de perfiles	Consulta (`GET /usuarios/me`) y edición (`PUT /usuarios/me`) de la información académica: nombre, carrera, universidad y teléfono de contacto.
RF-03	Publicación de ofertas	Gestión de los artículos que el estudiante tiene disponibles para intercambiar (título, materia, estado de conservación, fotografía).
RF-04	Publicación de búsquedas	Declaración de los materiales académicos que el estudiante necesita para el semestre en curso.
RF-05	Motor de matching	Algoritmo que correlaciona ofertas con búsquedas y sugiere trueques viables entre pares de estudiantes.
RF-06	Gestión del trueque	Máquina de estados de la negociación: propuesto → aceptado → completado, con transición alternativa a cancelado.
RF-07	Valoraciones post-trueque	Registro de calificaciones de 1 a 5 estrellas y comentario libre una vez finalizado el intercambio.

RF-08	Sistema de reputación	Cálculo del promedio de valoraciones y número de trueques completados, visible en cada publicación.
RF-09	Validación estricta de datos	Sanitización y validación automática de toda entrada mediante modelos Pydantic v2 antes de cualquier operación de persistencia.
RF-10	Administración y moderación	Roles diferenciados (`admin`, `moderador`, `user`) y promoción de usuarios por parte de un administrador.
RNF-01	Seguridad criptográfica	Hashing bcrypt con salt aleatorio por contraseña, JWT firmado HS256 con `jti` único y cookies con políticas `HttpOnly` / `SameSite`.
RNF-02	Rendimiento asíncrono	Consultas no bloqueantes con SQLAlchemy Async sobre el driver `asyncpg`, orientadas a soportar picos de concurrencia en semana de matrículas.
RNF-03	Mantenibilidad	Arquitectura hexagonal con dependencias apuntando hacia el dominio; estilo de código uniforme garantizado por Black y Ruff en `pre-commit`.
RNF-04	Portabilidad	Ejecución idéntica en local, Docker y servidor mediante contenedores y configuración externalizada en variables de entorno.
RNF-05	Verificabilidad	Suite de pruebas automatizadas unitarias y de integración ejecutables con un solo comando (`pytest -v`).
RNF-06	Observabilidad	Endpoints de *health check* (`/health` y `/health/db`) que permiten verificar en vivo el estado del servicio y de la conexión a base de datos.

1.6 Estado real de implementación

Un documento técnico honesto debe distinguir entre lo diseñado y lo construido. La siguiente matriz de trazabilidad refleja el estado verificable del código fuente a la fecha de esta entrega.

ID	Requerimiento	Estado	Evidencia / observación
RF-01	Registro e inicio de sesión	**Implementado**	`/auth/register`, `/auth/login`, `/auth/refresh`, `/auth/logout` · probado en `test_login_and_refresh`.
RF-02	Gestión de perfiles	**Implementado**	`GET/PUT /usuarios/me` · probado en `test_user_crud_and_promotion`.
RF-03	Publicación de ofertas	**Diseñado — pendiente**	Modelo de datos proyectado en §4.4. Sin router ni entidad en código.
RF-04	Publicación de búsquedas	**Diseñado — pendiente**	Comparte la entidad `publicaciones` mediante un discriminador de tipo.
RF-05	Motor de matching	**Diseñado — pendiente**	Depende de RF-03 y RF-04. Estrategia propuesta en §10.
RF-06	Gestión del trueque	**Diseñado — pendiente**	Máquina de estados definida; sin implementar.
RF-07	Valoraciones post-trueque	**Diseñado — pendiente**	Depende de RF-06.
RF-08	Sistema de reputación	**Diseñado — pendiente**	Campo agregado derivado de RF-07.
RF-09	Validación estricta	**Implementado**	Esquemas Pydantic v2 en `schemas/` aplicados en todos los routers activos.
RF-10	Administración y moderación	**Implementado**	`POST /admin/users/{id}/promote` con verificación de rol `admin` (403).
RNF-01 a RNF-06	No funcionales	**Implementado**	Verificables en el código, la suite de pruebas y los health checks.

> **Lectura del avance.** El 100 % de la suite de pruebas en verde certifica que *lo construido funciona correctamente* (8 de 8 pruebas del módulo de usuarios y autenticación), no que el sistema completo esté terminado. Sobre el total de requerimientos funcionales levantados, la cobertura actual es de 4 de 10; los 6 restantes se apoyan sobre el cimiento de identidad ya entregado.

2. Metodología de desarrollo y stack tecnológico

2.1 Metodología Scrumban

El equipo adoptó un enfoque ágil híbrido **Scrumban**, que combina la cadencia y las ceremonias de Scrum con la visualización de flujo y los límites de trabajo en curso de Kanban. Esta decisión responde a la realidad de un proyecto universitario: la disponibilidad del equipo es irregular (depende de parciales y entregas de otras asignaturas), por lo que un Scrum estricto con sprints de compromiso fijo genera deuda de planificación, mientras que un Kanban puro carece de los hitos de revisión que el cronograma académico exige.

Práctica	Aplicación concreta en el proyecto
Tablero Trello	Columnas *Backlog* → *Por hacer* → *En desarrollo* → *En revisión* → *Hecho*, con límite de trabajo en curso para evitar tareas abiertas simultáneas.
Sprints periódicos	Iteraciones con un objetivo verificable por entrega (p. ej. «autenticación JWT completa y probada»).

Definición de Hecho	Una tarea se cierra solo si: el código pasa Black y Ruff, existe prueba automatizada asociada, la suite completa queda en verde y la documentación se actualiza.
Revisión por pares	Integración mediante *pull requests* en GitHub; ninguna rama se fusiona a `main` sin revisión.
Retrospectiva	Al cierre de cada iteración se registran impedimentos técnicos y ajustes de estimación.

2.2 Stack tecnológico y justificación de cada elección

Toda decisión tecnológica se tomó con un criterio explícito y no por familiaridad. La siguiente tabla documenta qué se eligió, por qué, y cuál fue la alternativa descartada.

Componente	Tecnología	Justificación de la elección	Alternativa descartada
Framework web	FastAPI	Validación automática con tipos de Python, soporte nativo de `async/await` sobre ASGI y generación automática de documentación OpenAPI (Swagger y ReDoc) sin escribir una sola línea extra.	Django REST Framework (síncrono por defecto y más pesado para una API pura); Flask (requiere ensamblar validación y documentación a mano).
ORM	SQLAlchemy 2.0 Async	Estándar de facto en Python, con una API asíncrona madura y control fino del SQL generado cuando la consulta lo requiere.	Tortoise ORM (ecosistema menor); SQL crudo (mayor riesgo de inyección y de duplicación de lógica).
Driver de base de datos	asyncpg / aiosqlite	`asyncpg` es el driver PostgreSQL asíncrono más rápido de Python; `aiosqlite` permite ejecutar la misma capa de código sobre SQLite en pruebas.	`psycopg2` (bloqueante, anularía la ventaja del modelo asíncrono).
Motor relacional	PostgreSQL 15	Integridad referencial estricta, tipos avanzados, índices sobre texto para la búsqueda de material y madurez probada en producción.	MySQL (menor riqueza de tipos); MongoDB (el dominio del trueque es intrínsecamente relacional: usuario ↔ publicación ↔ trueque ↔ valoración).
Validación / DTOs	Pydantic v2	Núcleo escrito en Rust (validación notablemente más rápida que v1) e integración directa con FastAPI para la serialización de entrada y salida.	Marshmallow (sin integración nativa con el sistema de tipos de FastAPI).
Autenticación	PyJWT (HS256)	Tokens autocontenidos que evitan una consulta a base de datos en cada petición y permiten escalar horizontalmente sin sesiones compartidas.	Sesiones en servidor (requieren almacenamiento centralizado tipo Redis, innecesario en esta escala).
Hashing	bcrypt	Función de derivación deliberadamente lenta con factor de coste configurable y salt aleatorio por contraseña.	SHA-256 sin salt (vulnerable a tablas arcoíris y a ataques por GPU).
Migraciones	Alembic	Versionado del esquema de base de datos en el mismo repositorio que el código, con soporte para ejecución asíncrona.	`create_all()` en arranque (irrepetible, no versionable, inviable en producción).
Pruebas	Pytest + HTTPX + Respx	HTTPX con `ASGITransport` ejecuta la aplicación en memoria sin levantar servidor; Respx intercepta las llamadas HTTP salientes a Google para probar OAuth de forma determinista.	`unittest` (más verboso, sin fixtures asíncronas cómodas).
Contenerización	Docker + Compose	Reproduce el entorno completo (API + PostgreSQL con volumen persistente) con un único comando, eliminando el clásico «en mi máquina sí funciona».	Instalación manual de PostgreSQL por cada integrante del equipo.

2.3 Calidad de código: Black, Ruff y pre-commit

La uniformidad del código no se delega a la disciplina individual, sino que se automatiza mediante *hooks* de `pre-commit` que se ejecutan antes de confirmar cada cambio en Git. Ningún commit que incumpla el estilo llega al repositorio remoto.

```
repos:
  - repo: https://github.com/psf/black          # Formateo determinista del código
    rev: 24.4.2
    hooks: [{ id: black, language_version: python3.12 }]
  - repo: https://github.com/charliermarsh/ruff-pre-commit   # Linter (corrige automáticamente)
    rev: v0.5.0
    hooks: [{ id: ruff, args: [--fix] }]
  - repo: https://github.com/pre-commit/pre-commit-hooks
    rev: v5.0.0
    hooks: [{ id: trailing-whitespace }, { id: end-of-file-fixer }]
```

- **Black** impone un formato único e innegociable, lo que elimina las discusiones de estilo en las revisiones de código y reduce el ruido en los *diffs* de Git.
- **Ruff** detecta importaciones sin usar, variables muertas, comparaciones sospechosas y decenas de reglas más, corrigiendo automáticamente las que son seguras.
- Los hooks de `trailing-whitespace` y `end-of-file-fixer` normalizan los archivos y previenen conflictos de fusión triviales.

Activación en un equipo nuevo: `pip install pre-commit && pre-commit install`.

3. Arquitectura Hexagonal (Puertos y Adaptadores)

3.1 Motivación de la decisión arquitectónica

Se adoptó una **Arquitectura Hexagonal**, también conocida como Puertos y Adaptadores o Clean Architecture, cuyo principio rector es una única regla de dependencia: **las dependencias siempre apuntan hacia adentro, hacia el dominio**. El dominio —donde viven las reglas del trueque— no conoce FastAPI, no conoce PostgreSQL y no conoce PyJWT. Conoce únicamente interfaces abstractas que otros implementan por él.

Las consecuencias prácticas de esta decisión son concretas y verificables en este proyecto:

- **La base de datos es intercambiable.** La suite de pruebas corre sobre SQLite asíncrono mientras producción corre sobre PostgreSQL, sin modificar una sola línea del dominio: solo cambia el valor de `DATABASE_URL`.
- **La lógica de negocio es comprobable en aislamiento.** Las pruebas de `UserService` no necesitan levantar un servidor HTTP ni una base de datos real; basta con sustituir el repositorio por un doble de prueba.
- **La tecnología es reemplazable.** Migrar de PyJWT a otra librería de tokens, o de bcrypt a Argon2, implica escribir un nuevo adaptador y registrarlo, sin tocar el servicio de usuarios.
- **El crecimiento es predecible.** Cada módulo pendiente (publicaciones, trueques, valoraciones) se incorpora replicando el mismo patrón de cuatro capas, lo que hace el trabajo estimable y divisible entre integrantes.

3.2 Diagrama de capas

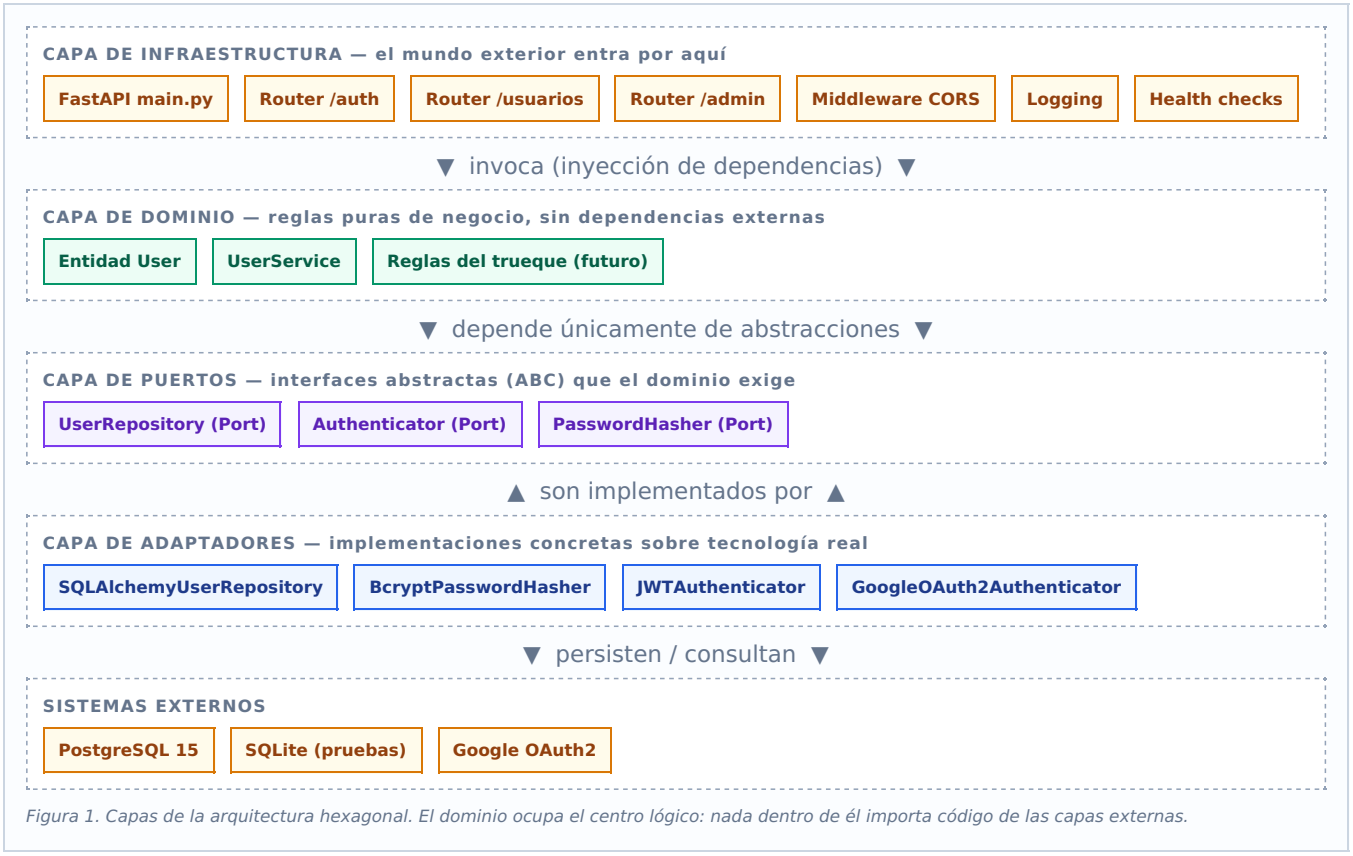


Figura 1. Capas de la arquitectura hexagonal. El dominio ocupa el centro lógico: nada dentro de él importa código de las capas externas.

3.3 Responsabilidad de cada capa

Capa	Responsabilidad principal	Archivos y componentes en el proyecto
Dominio	Contiene las entidades puras y las reglas de negocio de usuarios y trueques. No importa librerías de terceros.	`domain/models/user.py` `domain/services/user_service.py`
Puertos	Define mediante clases abstractas (ABC) el contrato que el dominio exige hacia el exterior: «necesito guardar un usuario», sin decir dónde.	`ports/repositories/user_repository.py` `ports/security/authenticator.py`

Adaptadores	Implementa los puertos conectando el dominio con la tecnología concreta elegida.	`adapters/db/sqlalchemy_user_repository.py`
		`adapters/security/password_hasher.py`
		`adapters/security/jwt_authenticator.py`
		`adapters/security/google_oauth.py`

Infraestructura	Provee el servidor FastAPI, los routers REST, el middleware CORS, el registro de logs y los endpoints de salud.	`infrastructure/main.py`
		`infrastructure/api/*`
		`infrastructure/security.py`

Configuración	Centraliza la lectura de variables de entorno con Pydantic Settings, tipada y validada al arranque.	`config/settings.py`
Esquemas	Define los DTO de entrada y salida de la API, desacoplando	`schemas/user.py`, `schemas/auth.py`

	el contrato público del modelo interno de dominio.	

3.4 Estructura de carpetas del proyecto

```
backend_estudiantil/
├── config/              # Configuración global con Pydantic Settings (settings.py)
│                        #   → lee .env y valida tipos al arrancar la aplicación
├── domain/              # NÚCLEO: reglas de negocio sin dependencias externas
│   ├── models/          #   → Entidad User (dataclass / objeto puro de Python)
│   └── services/        #   → UserService: crear, obtener, actualizar, eliminar
├── ports/               # Interfaces abstractas (ABC) que el dominio exige
│   ├── repositories/    #   → UserRepository: contrato de persistencia
│   └── security/        #   → Authenticator y PasswordHasher: contratos de seguridad
├── adapters/            # Implementaciones concretas de los puertos
│   ├── db/              #   → SQLAlchemyUserRepository, engine async, get_db()
│   └── security/        #   → BcryptPasswordHasher, JWTAuthenticator, GoogleOAuth2
├── infrastructure/      # Entrada HTTP: FastAPI y routers REST
│   ├── main.py          #   → instancia de la app, CORS, registro de routers
│   └── api/             #   → routers: auth.py, usuarios.py, admin.py, health.py
├── schemas/             # DTOs Pydantic v2 de entrada y salida
├── tests/               # Pruebas unitarias del dominio y CRUD del servicio
└── requirements.txt     # Dependencias fijadas del proyecto

alembic/                 # Versionado del esquema de base de datos
├── env.py               #   → configurado con async_engine_from_config
└── versions/1975ea83b712_crear_tabla_usuarios.py

tests/                   # Pruebas de integración de extremo a extremo
└── test_auth_integration.py

.env                     # Variables de entorno (DATABASE_URL, JWT_SECRET_KEY, ...)
.env.example             # Plantilla versionada sin secretos reales
Dockerfile               # Imagen Python 3.12-slim de la API
docker-compose.yml       # Orquestación: API + PostgreSQL 15 con volumen
alembic.ini              # Configuración de Alembic
.pre-commit-config.yaml  # Hooks de Black, Ruff y normalización de archivos
```

3.5 Inyección de dependencias

El mecanismo que une las capas sin acoplarlas es el sistema de dependencias de FastAPI (`Depends`). El router no construye sus colaboradores: los declara como parámetros y el framework los provee. Así, en una prueba se sustituye la dependencia real por un doble sin modificar el código productivo.

```python
# adapters/db/__init__.py — sesión asíncrona por petición
async def get_db() -> AsyncGenerator[AsyncSession, None]:
    async with AsyncSessionLocal() as session:
        try:
            yield session          # la sesión vive solo lo que dura la petición
            await session.commit()
        except Exception:
            await session.rollback()   # ante cualquier error, no se persiste nada
            raise
        finally:
            await session.close()

# infrastructure/api/usuarios.py — el router recibe, no construye
@router.get("/me", response_model=UserOut)
async def leer_perfil_propio(
    usuario_actual: User = Depends(obtener_usuario_actual),   # valida el JWT
    db: AsyncSession = Depends(get_db),                       # inyecta la sesión
):
    return usuario_actual
```

> **Por qué importa el `rollback` automático.** Si a mitad de una operación que toca varias tablas ocurre una excepción, la transacción se revierte completa. Aplicado al trueque: nunca quedará un intercambio marcado como completado si falló el registro de la valoración asociada.

3.6 Trazado de una petición de extremo a extremo

Para hacer tangible la arquitectura, se traza el recorrido completo de una petición de registro de estudiante, indicando la capa responsable en cada paso.

Paso	Capa	Qué ocurre
1	Cliente	El frontend envía `POST /auth/register` con un cuerpo JSON que contiene correo, contraseña, nombre, carrera y universidad.
2	Infraestructura	FastAPI recibe la petición, aplica el middleware CORS y la enruta al manejador correspondiente del router `auth`.
3	Esquemas	Pydantic v2 valida el cuerpo contra el DTO `UserCreate`. Si el correo no tiene formato válido o la contraseña es corta, la ejecución se detiene aquí con un **422** descriptivo y jamás alcanza el dominio.
4	Infraestructura	Se inyectan las dependencias: sesión de base de datos (`get_db`), repositorio y hasher.
5	Dominio	`UserService.crear_usuario()` aplica las reglas de negocio: verifica a través del puerto que el correo no esté

		registrado y construye la entidad `User` con rol por defecto `user`.
6	Adaptador de seguridad	`BcryptPasswordHasher` genera un salt aleatorio y devuelve el hash. La contraseña en texto plano nunca se almacena ni se registra en logs.
7	Adaptador de datos	`SQLAlchemyUserRepository` traduce la entidad de dominio al modelo ORM `UserORM` y ejecuta el `INSERT` asíncrono sobre la tabla `usuarios`.
8	Base de datos	PostgreSQL aplica las restricciones declaradas: unicidad del correo, no nulos y valores por defecto de `created_at` y `updated_at`.
9	Infraestructura	Se hace `commit` de la transacción y se serializa la respuesta con el DTO `UserOut`, que **excluye deliberadamente** el campo `hashed_password`.
10	Cliente	Recibe **201 Created** con el perfil público del estudiante recién registrado.

4. Base de datos y persistencia

4.1 Configuración visible de la conexión

La conexión a la base de datos es explícita y transparente: no está escondida en el código, sino externalizada en la variable de entorno `DATABASE_URL` dentro del archivo `.env`. Cambiar de motor es cambiar una línea.

```
# PostgreSQL — producción y Docker
DATABASE_URL=postgresql+asyncpg://postgres:123456@localhost:5432/trueque

# SQLite asíncrono — desarrollo local rápido y pruebas aisladas
# DATABASE_URL=sqlite+aiosqlite:///./test.db
```

La cadena se compone de cuatro partes que conviene saber leer: el **dialecto** (`postgresql`), el **driver asíncrono** (`+asyncpg`), las **credenciales y el host** (`postgres:123456@localhost:5432`) y el **nombre de la base** (`trueque`).

4.2 Motor asíncrono y sesión por petición

SQLAlchemy 2.0 en modo asíncrono permite que, mientras una consulta espera la respuesta de PostgreSQL, el bucle de eventos atienda otras peticiones en lugar de quedar bloqueado. Esto es determinante en el pico de uso esperado del sistema: la primera semana de clases, cuando cientos de estudiantes consultan material simultáneamente.

```
engine = create_async_engine(settings.DATABASE_URL, echo=False, future=True)

AsyncSessionLocal = async_sessionmaker(
    bind=engine,
    class_=AsyncSession,
    expire_on_commit=False,   # los objetos siguen utilizables después del commit
)
```

El endpoint `GET /health/db` ejecuta un ping real (`SELECT 1`) contra el motor configurado y reporta el estado de conectividad en vivo, lo que permite diagnosticar en segundos si un fallo proviene de la API o de la base de datos:

```
{
  "status": "ok",
  "database": {
    "status": "conectado",
    "dialecto": "postgresql",
    "driver": "asyncpg"
  }
}
```

4.3 Diccionario de datos: tabla `usuarios`

En concordancia con el requerimiento de nomenclatura del proyecto, la tabla se define formalmente en español como `usuarios` (modelo ORM `UserORM`), e incorpora los campos necesarios para la interacción universitaria.

Columna	Tipo de dato	Restricción / clave	Descripción en el negocio
`id`	VARCHAR (UUID)	PRIMARY KEY, INDEX	Identificador único global del estudiante. Se usa UUID en lugar de entero autoincremental para no exponer el número de usuarios registrados ni permitir enumeración de perfiles.
`email`	VARCHAR	UNIQUE, NOT NULL, INDEX	Correo electrónico institucional; es la credencial de inicio de sesión. El índice único garantiza que no existan cuentas duplicadas.
`hashed_password`	VARCHAR	NOT NULL	Hash bcrypt de la contraseña, con su salt embebido. Nunca se expone en ninguna respuesta de la API.
`name`	VARCHAR	NULLABLE	Nombre y apellidos del estudiante, mostrados en su perfil público y en las publicaciones.
`role`	VARCHAR	NOT NULL, DEFAULT `'user'`	Rol de control de acceso: `admin`, `moderador` o `user`. Determina qué endpoints puede invocar.
`is_active`	BOOLEAN	NOT NULL, DEFAULT TRUE	Estado de la cuenta. Permite el borrado lógico: desactivar a un

			usuario sin romper el historial de trueques y valoraciones que lo referencian.
`telefono`	VARCHAR	NULLABLE	Teléfono o WhatsApp para coordinar el punto de encuentro físico del intercambio dentro del campus.
`carrera`	VARCHAR	NULLABLE	Programa académico que cursa. Insumo clave para el motor de matching: los libros útiles suelen compartirse dentro de una misma carrera.
`universidad`	VARCHAR	NULLABLE	Universidad o campus donde se realizan los trueques. Permite acotar los resultados a la institución del estudiante.
`created_at`	DATETIME	NOT NULL, SERVER DEFAULT	Fecha y hora de registro en UTC, generada por el servidor de base de datos y no por el cliente.
`updated_at`	DATETIME	NOT NULL, SERVER DEFAULT	Fecha y hora de la última modificación del perfil; se actualiza en cada UPDATE.

Decisión de diseño: borrado lógico frente a borrado físico. El campo `is_active` existe porque eliminar físicamente a un usuario dejaría trueques huérfanos y valoraciones sin autor, corrompiendo el historial de reputación de su contraparte. Desactivar preserva la integridad referencial del sistema.

4.4 Modelo entidad-relación proyectado

La tabla `usuarios` es la única implementada en esta entrega. El siguiente modelo documenta el diseño completo previsto para los módulos pendientes, de forma que la siguiente iteración parta de un esquema ya razonado.

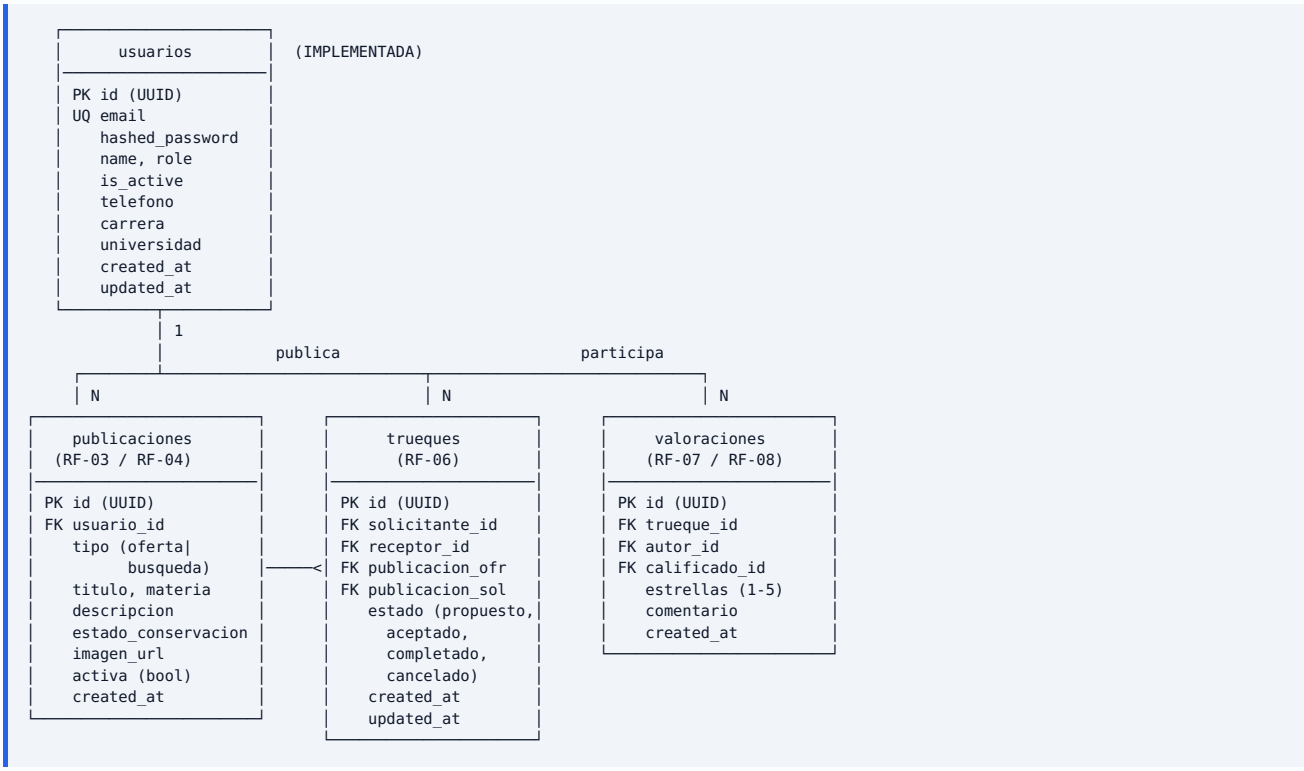


Figura 2. Modelo entidad-relación completo. Solo `usuarios` está implementada en la versión 1.0.0; las demás entidades constituyen el diseño aprobado para las siguientes iteraciones.

Notas de diseño sobre el modelo proyectado:

- **Una sola tabla para ofertas y búsquedas.** Ambas comparten estructura (título, materia, descripción); distinguirlas con un campo discriminador `tipo` evita duplicar tabla y consultas, y simplifica el motor de matching a una única consulta cruzada.
- **El trueque referencia dos publicaciones.** Lo ofrecido y lo solicitado, lo que deja registro histórico exacto de qué se intercambió aunque las publicaciones se desactiven después.
- **La valoración cuelga del trueque, no del usuario.** Así se garantiza la regla de negocio de que solo se puede calificar a alguien con quien efectivamente se completó un intercambio, impidiendo reseñas falsas.
- **La reputación (RF-08) es un valor derivado**, calculado como promedio de estrellas recibidas; no se almacena como campo editable para evitar inconsistencias.

4.5 Migraciones con Alembic

El esquema de la base de datos se versiona en el mismo repositorio que el código mediante Alembic. La migración inicial, nombrada en español, es `1975ea83b712_crear_tabla_usuarios.py`. El entorno `alembic/env.py` está configurado con `async_engine_from_config` para

ejecutar migraciones asíncronas.

```
# Generar una nueva migración detectando cambios en los modelos ORM
alembic revision --autogenerate -m "crear_tabla_publicaciones"

# Aplicar todas las migraciones pendientes hasta la última versión
alembic upgrade head

# Revertir la última migración aplicada
alembic downgrade -1

# Consultar la revisión activa en la base de datos
alembic current
```

Punto de mejora identificado. El archivo `alembic.ini` contiene la cadena de conexión con la contraseña escrita literalmente (`sqlalchemy.url = postgresql+asyncpg://postgres:123456@...`). Al estar versionado en Git, esa credencial queda expuesta en el historial del repositorio. La práctica recomendada es dejar `sqlalchemy.url` vacío en el archivo y que `env.py` la lea de la variable de entorno `DATABASE_URL`, como ya hace el resto de la aplicación.

5. Módulo de autenticación y seguridad

5.1 Hashing de contraseñas con bcrypt

Implementado en el adaptador `adapters/security/password_hasher.py` sobre la librería `bcrypt`. Cada contraseña se cifra con un salt criptográfico aleatorio generado por `bcrypt.gensalt()`.

```
class BcryptPasswordHasher(PasswordHasherPort):
    def hash(self, password: str) -> str:
        salt = bcrypt.gensalt()                      # salt aleatorio por contraseña
        return bcrypt.hashpw(password.encode(), salt).decode()

    def verify(self, password: str, hashed: str) -> bool:
        return bcrypt.checkpw(password.encode(), hashed.encode())
```

Tres propiedades hacen de bcrypt la elección correcta frente a un hash criptográfico genérico:

- **Salt único por usuario.** Dos estudiantes con la misma contraseña producen hashes completamente distintos, lo que invalida los ataques por tablas precalculadas (*rainbow tables*).
- **Lentitud deliberada.** A diferencia de SHA-256, diseñado para ser veloz, bcrypt está diseñado para ser costoso. Un atacante que obtenga la base de datos no puede probar millones de combinaciones por segundo.
- **Factor de coste ajustable.** Conforme el hardware se vuelve más rápido, el parámetro de coste se incrementa sin cambiar de algoritmo ni migrar los hashes existentes.

El hash resultante es autodescriptivo y contiene toda la información necesaria para verificarlo:

```
$2b$12$Kx7LqP9vN2mR8sT4uW6yBe1cD3fG5hJ7kL9nM2pQ4rS6tU8vW0xYz
└┘└┘└┘ └──────────────────────┘└─────────────────────────────┘
 │  │   │                      └── salt (22 car.) + hash (31 car.)
 │  │   └── factor de coste: 2^12 = 4096 iteraciones
 │  └── identificador del algoritmo bcrypt
```

5.2 Tokens JWT y el identificador único JTI

La autenticación se basa en JSON Web Tokens firmados con el algoritmo **HS256** y una clave secreta configurable. Un JWT es autocontenido: lleva dentro la identidad y el rol del usuario, por lo que el servidor puede autorizar una petición sin consultar la base de datos.

Token	Vida útil	Dónde viaja	Propósito
Access Token	30 minutos	Cabecera `Authorization: Bearer <token>`	Autoriza cada petición a rutas protegidas. Su vida corta limita la ventana de daño si es interceptado.
Refresh Token	7 días	Cookie `HttpOnly`	Permite obtener un nuevo access token sin pedir la contraseña de nuevo, mediante `POST /auth/refresh`.

Estructura del *payload* del access token:

```
{
  "sub":  "3f9a1c77-8b2e-4d61-9f0a-7c5e2b4d8a13",   // identificador del estudiante
  "role": "user",                                    // rol para el control de acceso
  "exp":  1789123456,                                // instante de expiración (UNIX)
  "iat":  1789121656,                                // instante de emisión
  "jti":  "a7f3c2e1-4b8d-49f6-a1c3-9e7b5d2f8a04"     // identificador único del token
}
```

> **¿Para qué sirve el `jti`?** El campo `exp` tiene resolución de un segundo. Sin un identificador adicional, dos tokens emitidos para el mismo usuario dentro del mismo segundo serían bytes idénticos. El `jti` (un UUID4, conforme a RFC 7519) garantiza unicidad y habilita una futura lista de revocación: invalidar un token concreto —por ejemplo, tras un reporte de robo de cuenta— sin obligar a todos los usuarios a iniciar sesión de nuevo.

5.3 Cookies HttpOnly para el refresh token

El refresh token nunca se envía en el cuerpo de la respuesta: se deposita en una cookie del navegador con directivas de seguridad explícitas.

Directiva	Valor	Protección que aporta
`HttpOnly`	True	Impide que cualquier script JavaScript lea la cookie. Es la mitigación central contra el robo de sesión por **XSS**: aunque un atacante logre inyectar un script, no puede extraer el token.
`SameSite`	Lax	El navegador no envía la cookie en peticiones originadas desde otros sitios, bloqueando ataques de falsificación de petición (**CSRF**).
`Secure`	Según entorno	`False` en desarrollo local sobre HTTP; `True` en producción, donde obliga a transmitir la cookie exclusivamente por HTTPS.
`Max-Age`	7 días	Alinea la caducidad de la cookie con la del propio token, evitando cookies zombis en el navegador.

```
response.set_cookie(
    key="refresh_token",
    value=refresh_token,
    httponly=settings.COOKIE_HTTPONLY,     # True
    samesite=settings.COOKIE_SAMESITE,     # "lax"
    secure=settings.COOKIE_SECURE,         # False en local, True en producción
    max_age=settings.REFRESH_TOKEN_EXPIRE_DAYS * 24 * 60 * 60,
)
```

El endpoint `POST /auth/logout` elimina activamente esta cookie del navegador, cerrando la sesión de forma inmediata.

5.4 Flujo completo de autenticación

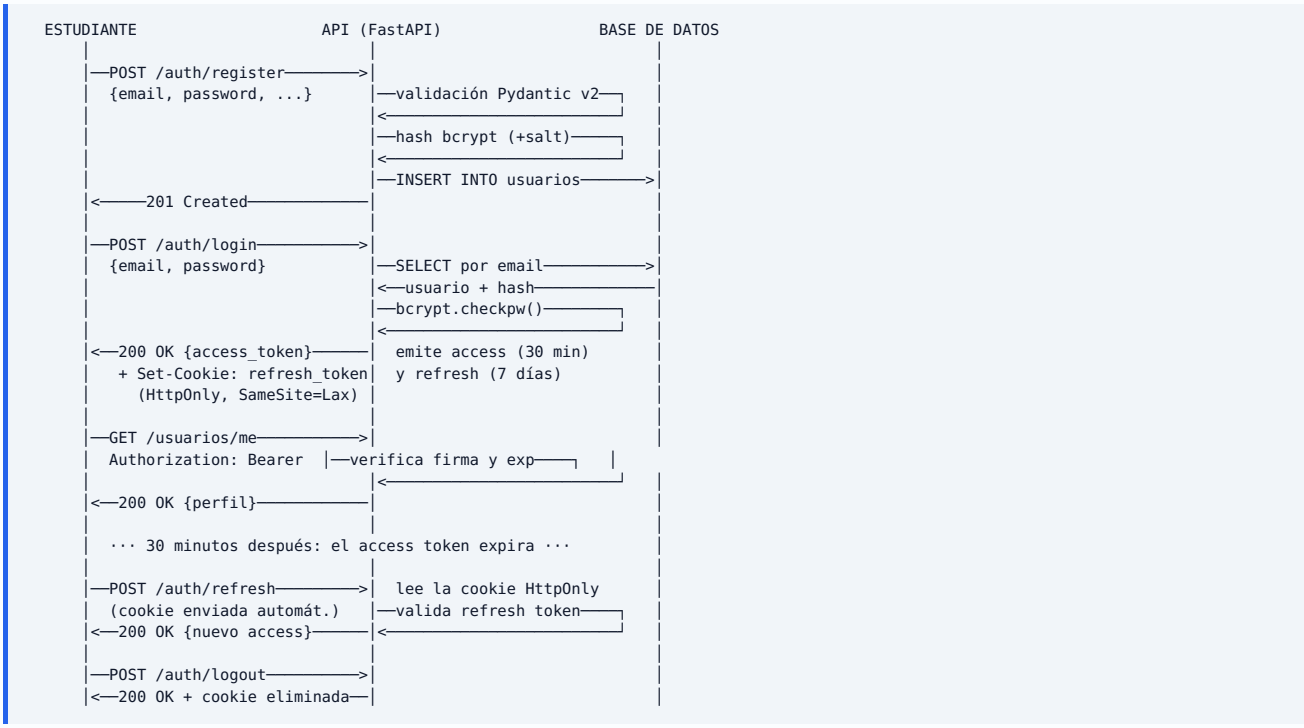


Figura 3. Ciclo de vida completo de la sesión: registro, inicio de sesión, uso autorizado, renovación silenciosa y cierre.

5.5 Control de acceso basado en roles (RBAC)

Cada usuario porta un rol en la columna `role` y en el *claim* homónimo del JWT. Las dependencias de FastAPI verifican ese rol antes de ejecutar el manejador, devolviendo **403 Forbidden** cuando los permisos son insuficientes.

Rol	Permisos	Justificación en el negocio
`user`	Gestionar su propio perfil, publicar, proponer y	Rol por defecto de todo estudiante registrado. Es el usuario natural

	aceptar trueques, valorar a sus contrapartes.	del sistema.
`moderador`	Todo lo anterior, más revisar y retirar publicaciones reportadas como inapropiadas.	Permite el control de contenido sin ceder acceso administrativo total, típicamente asignado a representantes estudiantiles.
`admin`	Acceso completo: crear, actualizar y eliminar cualquier usuario, y promover a moderador.	Reservado al equipo responsable de la plataforma.

```
async def requerir_admin(usuario: User = Depends(obtener_usuario_actual)) -> User:
    if usuario.role != "admin":
        raise HTTPException(status_code=403, detail="Permisos insuficientes")
    return usuario

@router.post("/users/{user_id}/promote")
async def promover_a_moderador(user_id: str, _: User = Depends(requerir_admin)):
    ...
```

5.6 Inicio de sesión federado con Google OAuth2

Para reducir la fricción del registro, el sistema soporta autenticación delegada con Google, especialmente pertinente cuando la universidad provee correos institucionales gestionados por Google Workspace.

1. El cliente solicita `GET /auth/google/login`, que devuelve la URL de consentimiento de Google.
2. El estudiante autoriza en el dominio de Google; la plataforma nunca ve su contraseña.
3. Google redirige a `GET /auth/google/callback` con un código de autorización de un solo uso.
4. El adaptador `GoogleOAuth2Authenticator` intercambia ese código por el perfil del usuario.
5. Si el correo no existe en `usuarios`, se crea la cuenta automáticamente; si existe, se reutiliza.
6. Se emiten el access token y la cookie de refresh exactamente igual que en el inicio de sesión tradicional.

Este flujo está cubierto por la prueba automatizada `test_google_oauth_flow`, que usa **Respx** para simular las respuestas de Google y validar el intercambio sin depender de la red.

5.7 Riesgos identificados y recomendaciones

Un análisis de seguridad riguroso debe señalar lo que falta, no solo lo que se hizo bien. Los siguientes puntos son el plan de endurecimiento previo a un despliegue real con usuarios.

Hallazgo	Severidad	Recomendación
Clave JWT y contraseña de PostgreSQL literales en archivos del repositorio (`.env`, `alembic.ini`, `docker-compose.yml`)	**Alta**	Versionar únicamente `.env.example` con valores ficticios; añadir `.env` a `.gitignore`; generar el secreto con `secrets.token_urlsafe(64)` y rotarlo para producción. Si ya fue confirmado en Git, debe considerarse comprometido y reemplazarse.
Ausencia de limitación de intentos (*rate limiting*) en `/auth/login`	**Alta**	Aplicar `slowapi` u otro limitador por dirección IP y por correo, con bloqueo temporal progresivo tras varios intentos fallidos consecutivos.
No se verifica la propiedad del correo institucional	Media	Enviar un enlace de confirmación al registrarse y restringir el dominio del correo al de la universidad, condición necesaria para la confianza entre pares.
Los refresh tokens no son revocables	Media	Almacenar el `jti` de los tokens emitidos y permitir su invalidación en el cierre de sesión o ante un reporte de cuenta comprometida.
Política de contraseñas no documentada	Media	Exigir longitud mínima y complejidad desde el validador Pydantic, y contrastar contra listas de contraseñas filtradas.
CORS potencialmente permisivo en desarrollo	Media	Declarar en producción una lista blanca explícita de orígenes del frontend, nunca `allow_origins=["*"]` junto con credenciales.
Contraseña por defecto 123456 en la base de datos de Docker	Media	Parametrizar `POSTGRES_PASSWORD` desde variables de entorno del host y usar un valor robusto fuera del entorno de desarrollo.

6. Contratos de datos: esquemas Pydantic v2

Los esquemas (DTO, *Data Transfer Objects*) son el contrato público de la API y cumplen dos funciones críticas: validar y sanear todo lo que entra, y controlar con precisión qué sale. Desacoplar el DTO de la entidad de dominio es lo que permite, por ejemplo, que la tabla tenga `hashed_password` mientras la respuesta jamás lo incluye.

Esquema	Dirección	Campos	Uso y validaciones
`UserCreate`	Entrada	email, password, name, carrera, universidad, telefono	Registro de estudiante. `EmailStr` valida el formato del correo; la contraseña exige longitud mínima.
`UserLogin`	Entrada	email, password	Credenciales de inicio de sesión.
`UserUpdate`	Entrada	name, telefono, carrera, universidad (todos opcionales)	Actualización parcial del perfil: solo se modifican los campos efectivamente enviados.

`UserOut`	Salida	id, email, name, role, is_active, telefono, carrera, universidad, created_at	Perfil público. **Excluye deliberadamente** `hashed_password`. Configurado con `from_attributes=True` para serializar directamente desde el objeto ORM.
`Token`	Salida	access_token, token_type	Respuesta de `/auth/login` y `/auth/refresh`. El refresh token viaja en cookie, no en este cuerpo.
`TokenPayload`	Interno	sub, role, exp, iat, jti	Representación tipada del contenido del JWT una vez decodificado.

```
class UserCreate(BaseModel):
    email: EmailStr                                  # valida formato de correo
    password: str = Field(min_length=8)              # longitud mínima obligatoria
    name: str | None = None
    carrera: str | None = None
    universidad: str | None = None
    telefono: str | None = None

class UserOut(BaseModel):
    model_config = ConfigDict(from_attributes=True)    # serializa desde el ORM
    id: str
    email: EmailStr
    name: str | None
    role: str
    is_active: bool
    carrera: str | None
    universidad: str | None
    telefono: str | None
    created_at: datetime
    # hashed_password NO se declara: es imposible que se filtre en una respuesta
```

Beneficio colateral. FastAPI deriva de estos mismos esquemas el documento OpenAPI, de modo que la documentación interactiva en `/docs` y `/redoc` siempre está sincronizada con el código: es imposible que la documentación de la API quede desactualizada respecto de su implementación.

7. Catálogo completo de endpoints REST

7.1 Matriz de rutas

Método	Ruta	Acceso / rol	Propósito funcional
POST	`/auth/register`	Público	Registra un estudiante con sus datos académicos y responde **201 Created**.
POST	`/auth/login`	Público	Valida las credenciales con bcrypt y emite access token más cookie HttpOnly de refresh.
POST	`/auth/refresh`	Público (cookie)	Renueva el access token a partir de la cookie de refresh, sin pedir contraseña.
POST	`/auth/logout`	Autenticado	Cierra la sesión y elimina la cookie de refresh del navegador.
GET	`/auth/google/login`	Público	Genera y retorna la URL de consentimiento de Google OAuth2.
GET	`/auth/google/callback`	Público	Recibe el código de Google, crea al estudiante si no existe y emite los tokens.
GET	`/usuarios/me`	Estudiante (token)	Retorna el perfil completo del usuario en sesión: carrera, universidad, teléfono y rol.
PUT	`/usuarios/me`	Estudiante (token)	Actualiza nombre, teléfono, carrera o universidad del propio estudiante.
GET	`/usuarios/`	Autenticado	Listado paginado de usuarios mediante los parámetros `skip` y `limit`.
GET	`/usuarios/{user_id}`	Autenticado	Obtiene el detalle de un usuario específico; responde **404** si no existe.
POST	`/usuarios/`	Solo admin	Creación administrativa directa de usuarios en la plataforma.
PUT	`/usuarios/{user_id}`	Admin o propietario	Actualización de la información de un usuario por parte de un administrador o del propio interesado.
DELETE	`/usuarios/{user_id}`	Solo admin	Elimina un usuario del sistema; responde **204 No Content**.
POST	`/admin/users/{id}/promote`	Solo admin	Promueve a un usuario al rol privilegiado de `moderador`.
GET	`/health`	Público	Verificación básica de que el servidor FastAPI responde.
GET	`/health/db`	Público	Verificación profunda: ejecuta un ping real contra PostgreSQL o SQLite.

7.2 Ejemplos de petición y respuesta

POST /auth/register — Registro de estudiante

```
PETICIÓN
POST /auth/register
Content-Type: application/json

{
  "email": "laura.gomez@universidad.edu.co",
  "password": "TruequeSeguro2026",
  "name": "Laura Gómez Restrepo",
  "carrera": "Ingeniería de Sistemas",
  "universidad": "Universidad Nacional",
  "telefono": "3001234567"
}

RESPUESTA  201 Created
{
  "id": "3f9a1c77-8b2e-4d61-9f0a-7c5e2b4d8a13",
  "email": "laura.gomez@universidad.edu.co",
  "name": "Laura Gómez Restrepo",
  "role": "user",
  "is_active": true,
  "carrera": "Ingeniería de Sistemas",
  "universidad": "Universidad Nacional",
  "telefono": "3001234567",
  "created_at": "2026-09-16T14:22:08.441Z"
}
```

POST /auth/login — Inicio de sesión

```
PETICIÓN
POST /auth/login
Content-Type: application/json

{ "email": "laura.gomez@universidad.edu.co", "password": "TruequeSeguro2026" }

RESPUESTA  200 OK
Set-Cookie: refresh_token=eyJhbGciOiJIUzI1NiIs...; HttpOnly; SameSite=Lax; Max-Age=604800

{
  "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIzZjlhMWM3Ny04...",
  "token_type": "bearer"
}
```

GET /usuarios/me — Perfil del estudiante autenticado

```
PETICIÓN
GET /usuarios/me
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...

RESPUESTA  200 OK
{
  "id": "3f9a1c77-8b2e-4d61-9f0a-7c5e2b4d8a13",
  "email": "laura.gomez@universidad.edu.co",
  "name": "Laura Gómez Restrepo",
  "role": "user",
  "is_active": true,
  "carrera": "Ingeniería de Sistemas",
  "universidad": "Universidad Nacional",
  "telefono": "3001234567",
  "created_at": "2026-09-16T14:22:08.441Z"
}
```

PUT /usuarios/me — Actualización parcial del perfil

```
PETICIÓN
PUT /usuarios/me
Authorization: Bearer <access_token>

{ "telefono": "3109876543", "carrera": "Ingeniería Industrial" }

RESPUESTA  200 OK   (solo cambian los campos enviados; el resto se conserva)
{ "id": "3f9a1c77-...", "carrera": "Ingeniería Industrial", "telefono": "3109876543", ... }
```

GET /usuarios/ — Listado paginado

```
PETICIÓN
GET /usuarios/?skip=0&limit=10
Authorization: Bearer <access_token>

RESPUESTA  200 OK
[
  { "id": "3f9a1c77-...", "name": "Laura Gómez Restrepo",  "carrera": "Ing. de Sistemas"  },
  { "id": "7b2d4e91-...", "name": "Andrés Vélez Muñoz",    "carrera": "Ing. Industrial"   }
]
```

Errores representativos

```
401 Unauthorized — credenciales incorrectas
{ "detail": "Correo o contraseña incorrectos" }

403 Forbidden — un usuario común intenta promover a otro
{ "detail": "Permisos insuficientes" }

404 Not Found — identificador inexistente
{ "detail": "Usuario no encontrado" }

422 Unprocessable Entity — validación de Pydantic v2
{
  "detail": [
    { "type": "value_error", "loc": ["body", "email"],
      "msg": "value is not a valid email address" }
  ]
}
```

7.3 Manejo estándar de errores

Código	Significado	Cuándo lo devuelve este backend
200 OK	Operación exitosa	Consultas, inicio de sesión, renovación de token y actualizaciones correctas.
201 Created	Recurso creado	Registro exitoso de un nuevo estudiante.
204 No Content	Éxito sin cuerpo	Eliminación de un usuario por parte de un administrador.
400 Bad Request	Solicitud inválida	Correo ya registrado u operación que viola una regla de negocio del dominio.
401 Unauthorized	No autenticado	Credenciales incorrectas, token ausente, firma inválida o token expirado.
403 Forbidden	Sin permisos	El usuario está autenticado pero su rol no alcanza para la acción solicitada.
404 Not Found	No encontrado	Identificador de usuario inexistente en la base de datos.
422 Unprocessable Entity	Validación fallida	El cuerpo no cumple el esquema Pydantic: tipo incorrecto, campo faltante o formato inválido.
500 Internal Server Error	Error no controlado	Excepción inesperada; la transacción se revierte automáticamente y el detalle se registra en el log, nunca se expone al cliente.

> **Distinción entre 400 y 422.** Un **422** indica que la petición ni siquiera tiene la forma correcta (falla el contrato de datos, lo detecta Pydantic antes de llegar al dominio). Un **400** significa que la petición está bien formada pero infringe una regla de negocio, como intentar registrar un correo que ya existe.

8. Pruebas automatizadas y aseguramiento de calidad

La suite de pruebas se ejecuta con **Pytest**, **HTTPX** y **Respx**. Todas las pruebas corren sobre una base de datos **SQLite asíncrona aislada**, de modo que ninguna ejecución compromete ni contamina los datos de PostgreSQL. El transporte `ASGITransport` de HTTPX permite invocar la aplicación en memoria, sin levantar un servidor real: las pruebas de integración son casi tan rápidas como las unitarias.

8.1 Estrategia de pruebas por nivel

Nivel	Qué verifica	Archivo y técnica
Unitaria de dominio	Las reglas de negocio del servicio de usuarios, con el repositorio sustituido por un doble de prueba.	`test_user_service.py` — es posible gracias a que el dominio depende de puertos abstractos, no de SQLAlchemy.
Integración de CRUD	El flujo completo de extremo a extremo sobre la API real: registro, login, perfil, actualización, listado y promoción de rol.	`test_user_crud.py` — HTTPX con `ASGITransport`.
Integración de autenticación	Emisión de tokens, cookie HttpOnly, renovación, cierre de sesión y salud de la base de datos.	`test_auth_integration.py`
Servicio externo simulado	El intercambio OAuth2 con Google sin tocar la red, con respuestas controladas y deterministas.	`test_google_oauth_flow` — intercepción con Respx.

8.2 Casos de prueba y resultados

Archivo	Función de prueba	Resultado	Aspecto validado
`test_user_service.py`	`test_create_user`	**PASSED**	Creación de usuario en el dominio y hasheo seguro de la

			contraseña.
test_user_service.py	test_get_user	**PASSED**	Búsqueda por identificador y validación de los campos de la entidad.
test_user_service.py	test_update_user	**PASSED**	Actualización de nombre y datos en el servicio de dominio.
test_user_service.py	test_delete_user	**PASSED**	Eliminación y captura de `ValueError` al consultar un registro ya borrado.
test_user_crud.py	test_user_crud_and_promotion	**PASSED**	Flujo E2E: registro, login, `/me`, actualización de `/me`, listado y ascenso a moderador.
test_auth_integration.py	test_health_and_database_connection	**PASSED**	Respuesta de `/health` y conexión real verificada en `/health/db`.
test_auth_integration.py	test_login_and_refresh	**PASSED**	Emisión del access token, cookie HttpOnly, renovación y cierre de sesión.
test_auth_integration.py	test_google_oauth_flow	**PASSED**	Intercambio de código OAuth simulado con Respx y emisión correcta del JWT.

8.3 Salida de la ejecución

```
$ pytest -v

============================= test session starts =============================
platform win32 -- Python 3.12.3, pytest-9.0.3, pluggy-1.6.0
plugins: anyio-4.3.0, asyncio-1.4.0, respx-0.23.1

backend_estudiantil/tests/test_user_crud.py::test_user_crud_and_promotion PASSED  [ 12%]
backend_estudiantil/tests/test_user_service.py::test_create_user          PASSED  [ 25%]
backend_estudiantil/tests/test_user_service.py::test_get_user             PASSED  [ 37%]
backend_estudiantil/tests/test_user_service.py::test_update_user          PASSED  [ 50%]
backend_estudiantil/tests/test_user_service.py::test_delete_user          PASSED  [ 62%]
tests/test_auth_integration.py::test_health_and_database_connection       PASSED  [ 75%]
tests/test_auth_integration.py::test_login_and_refresh                    PASSED  [ 87%]
tests/test_auth_integration.py::test_google_oauth_flow                    PASSED  [100%]

============================== 8 passed in 3.56s ==============================
```

8.4 Casos aún no cubiertos

La suite verifica exhaustivamente los caminos exitosos. Para elevar la calidad del aseguramiento se recomienda incorporar pruebas de los caminos de error, que son donde suelen esconderse los defectos:

- Registro con un correo ya existente (debe responder **400**).
- Inicio de sesión con contraseña incorrecta (debe responder **401** y no revelar si el correo existe).
- Acceso a `/usuarios/me` sin token o con un token manipulado o expirado (**401**).
- Usuario con rol `user` intentando eliminar a otro usuario (**403**).
- Consulta de un identificador inexistente (**404**).
- Medición de cobertura con `pytest --cov=backend_estudiantil` para cuantificar el porcentaje de código ejercitado.

9. Despliegue y puesta en marcha

9.1 Requisitos del entorno

- **Python 3.12** o superior.
- **PostgreSQL 15** o superior (o SQLite local para una prueba rápida sin instalar el motor).
- **Docker y Docker Compose** (opcional, recomendado para reproducir el entorno completo).
- **Git** para el control de versiones y la activación de los hooks de `pre-commit`.

9.2 Despliegue local paso a paso

```
# 1. Clonar el repositorio
git clone <url-del-repositorio> && cd backendEstudiantil

# 2. Crear y activar un entorno virtual aislado
python -m venv .venv
source .venv/bin/activate          # Linux / macOS
.venv\Scripts\activate             # Windows

# 3. Instalar las dependencias fijadas
pip install -r backend_estudiantil/requirements.txt

# 4. Configurar las variables de entorno
cp backend_estudiantil/.env.example .env
#    editar .env y ajustar DATABASE_URL y JWT_SECRET_KEY

# 5. Aplicar las migraciones del esquema
alembic upgrade head

# 6. Ejecutar la API con recarga en vivo
uvicorn backend_estudiantil.infrastructure.main:app --host 0.0.0.0 --port 8000 --reload

# 7. Verificar el despliegue
#    Swagger UI .......... http://localhost:8000/docs
#    ReDoc ............... http://localhost:8000/redoc
#    Salud del servicio .. http://localhost:8000/health
#    Salud de la BD ...... http://localhost:8000/health/db
```

9.3 Despliegue con Docker Compose

El proyecto incluye un `Dockerfile` basado en `python:3.12-slim` y un `docker-compose.yml` que orquesta la API junto con PostgreSQL 15 y un volumen persistente para los datos. Un único comando levanta todo el entorno:

```
docker compose up --build
```

Servicio	Imagen	Puerto	Observaciones
db	postgres:15	5432	Base `trueque`, con volumen `pg_data` montado en `/var/lib/postgresql/data` para que los datos sobrevivan al reinicio del contenedor.
api	build local	8000	Lee su configuración de `.env`, declara `depends_on: db` y se reinicia automáticamente (`restart: unless-stopped`).

Detalles relevantes del `Dockerfile`: se establece `PYTHONDONTWRITEBYTECODE=1` y `PYTHONUNBUFFERED=1` para que los logs salgan en tiempo real, y se copia primero `requirements.txt` antes que el resto del código, aprovechando la caché de capas de Docker: cambiar una línea de código no obliga a reinstalar todas las dependencias.

> **Ajuste necesario para Docker.** Dentro de la red de Compose, el contenedor de la API no alcanza la base de datos por `localhost`, sino por el nombre del servicio. La variable debe quedar como `postgresql+asyncpg://postgres:<clave>@db:5432/trueque`, sustituyendo `localhost` por `db`.

9.4 Variables de entorno

Variable	Valor de ejemplo	Propósito
`DATABASE_URL`	`postgresql+asyncpg://...`	Cadena de conexión completa al motor de base de datos.
`JWT_SECRET_KEY`	(secreto generado)	Clave de firma de los tokens. Quien la posea puede falsificar sesiones: es el secreto más crítico del sistema.
`JWT_ALGORITHM`	`HS256`	Algoritmo de firma simétrica de los JWT.
`ACCESS_TOKEN_EXPIRE_MINUTES`	`30`	Vida útil del access token en minutos.
`REFRESH_TOKEN_EXPIRE_DAYS`	`7`	Vida útil del refresh token en días.
`COOKIE_SECURE`	`False` / `True`	`False` en desarrollo sobre HTTP; **obligatoriamente** `True` en producción con HTTPS.
`COOKIE_HTTPONLY`	`True`	Bloquea el acceso a la cookie desde JavaScript.
`COOKIE_SAMESITE`	`lax`	Política de envío de la cookie entre sitios.
`GOOGLE_CLIENT_ID`	(credencial de Google)	Identificador de la aplicación en Google Cloud Console.
`GOOGLE_CLIENT_SECRET`	(credencial de Google)	Secreto asociado al cliente OAuth2.
`GOOGLE_REDIRECT_URI`	`http://localhost:8000/auth/google/callback`	URL de retorno registrada en Google; debe coincidir exactamente.

10. Limitaciones actuales y trabajo futuro

10.1 Limitaciones reconocidas de la versión 1.0.0

- Solo está implementado el módulo de usuarios y autenticación. El núcleo funcional del trueque (publicaciones, matching, negociación y valoraciones) está diseñado pero no construido.
- No existe verificación de la propiedad del correo institucional, por lo que cualquier dirección con formato válido puede registrarse.
- No hay limitación de intentos de inicio de sesión, lo que deja el sistema expuesto a ataques de fuerza bruta.
- Los refresh tokens no pueden revocarse antes de su expiración natural.
- No se gestiona la carga de imágenes, indispensable para que una publicación de un libro resulte creíble.
- No hay notificaciones: un estudiante no se entera de una propuesta de trueque a menos que entre a la plataforma.
- Las pruebas cubren los caminos exitosos, pero no los de error ni los casos límite.

10.2 Hoja de ruta propuesta

Iteración	Objetivo	Entregables concretos
Sprint 2	Módulo de publicaciones (RF-03 y RF-04)	Entidad `Publicacion`, puerto `PublicacionRepository`, adaptador SQLAlchemy, router CRUD, migración Alembic y pruebas asociadas. Búsqueda por materia y por título con índice de texto.
Sprint 3	Motor de matching (RF-05)	Consulta que cruza las ofertas activas contra las búsquedas del estudiante, priorizando coincidencias dentro de la misma universidad y carrera. Endpoint `GET /matching/sugerencias`.
Sprint 4	Gestión del trueque (RF-06)	Máquina de estados con transiciones validadas en el dominio: solo el receptor puede aceptar, solo ambas partes pueden marcar como completado, y no se admite retroceder de estado.
Sprint 5	Valoraciones y reputación (RF-07 y RF-08)	Registro de estrellas y comentario condicionado a un trueque completado; cálculo del promedio expuesto en el perfil y en cada publicación.
Sprint 6	Endurecimiento y despliegue	Rate limiting, verificación de correo, revocación de tokens, carga de imágenes, notificaciones y publicación en un servidor real con HTTPS.

10.3 Mejoras técnicas recomendadas

- **Integración continua.** Un flujo de GitHub Actions que ejecute Black, Ruff y la suite completa en cada *pull request*, impidiendo fusionar código que rompa las pruebas.
- **Versionado de la API.** Prefijar las rutas con `/api/v1` para poder evolucionar el contrato sin romper a los clientes ya desplegados.
- **Paginación con metadatos.** Devolver `{ items, total, skip, limit }` en lugar de una lista simple, para que el frontend pueda construir controles de navegación.
- **Logging estructurado.** Registrar en formato JSON con identificador de correlación por petición, facilitando la trazabilidad de incidentes.
- **Índices adicionales.** Sobre `universidad` y `carrera`, que serán los filtros más frecuentes del motor de matching.
- **Cobertura de pruebas medida.** Incorporar `pytest-cov` y fijar un umbral mínimo como criterio de aceptación.

11. Conclusiones

El backend de la Plataforma de Trueque Estudiantil entrega, en su versión 1.0.0, un **módulo de identidad y autenticación completo, probado y documentado**, construido sobre una arquitectura hexagonal que separa con rigor las reglas de negocio de la tecnología que las soporta. Las ocho pruebas automatizadas de la suite se ejecutan en verde y certifican que el registro, el inicio de sesión, la renovación de tokens, la gestión de perfiles, el control de roles y la conexión a la base de datos funcionan de extremo a extremo.

El principal aporte técnico del proyecto no es un endpoint concreto, sino la **disciplina estructural** adoptada. La misma decisión que permite hoy ejecutar toda la suite sobre SQLite mientras producción corre sobre PostgreSQL —sin tocar una línea del dominio— es la que permitirá incorporar publicaciones, matching y valoraciones replicando un patrón ya validado, en lugar de improvisar sobre código acoplado. En un proyecto universitario donde el equipo rota y el tiempo es escaso, esa previsibilidad tiene un valor superior al de cualquier funcionalidad individual.

El sistema no está terminado, y este documento lo declara explícitamente: seis de los diez requerimientos funcionales permanecen en fase de diseño, y el análisis de seguridad de la sección 5.7 identifica hallazgos que deben resolverse antes de exponer la plataforma a usuarios reales. Reconocer con precisión ese límite —y dejar documentados el modelo de datos, la hoja de ruta y las correcciones pendientes— es parte del trabajo de ingeniería, no una deuda de él.

Dictamen: el componente entregado cumple los requerimientos de seguridad, integridad, rendimiento asíncrono y verificabilidad definidos para su alcance, y constituye una base sólida y auditable para la conexión inmediata con el cliente frontend y para las iteraciones posteriores.

12. Glosario de términos técnicos

Término	Definición
Adaptador	Clase que implementa un puerto conectando el dominio con una tecnología concreta (por ejemplo, SQLAlchemy o bcrypt).
Alembic	Herramienta de versionado del esquema de base de datos: cada cambio queda registrado como una migración reproducible y reversible.
API REST	Interfaz que expone recursos mediante URLs y verbos HTTP (GET, POST, PUT, DELETE) sobre un protocolo sin estado.
ASGI	Estándar de interfaz entre servidor y aplicación Python que soporta ejecución asíncrona; sucesor de WSGI.
bcrypt	Función de derivación de claves con salt y coste computacional ajustable, diseñada específicamente para almacenar contraseñas.
CORS	Mecanismo por el cual el navegador autoriza o bloquea peticiones desde un origen distinto al del servidor.
CSRF	Ataque que induce al navegador de una víctima autenticada a ejecutar acciones no deseadas; se mitiga con `SameSite`.
DTO	Objeto de transferencia de datos; define qué entra y qué sale de la API, separado del modelo interno de dominio.
Dominio	Capa que contiene las reglas de negocio puras del sistema, sin dependencias de frameworks ni de bases de datos.
E2E	Prueba de extremo a extremo que recorre el flujo completo tal como lo haría un usuario real.
Health check	Endpoint que reporta si el servicio y sus dependencias están operativos.
HttpOnly	Directiva de cookie que impide su lectura desde JavaScript, mitigando el robo de sesión por XSS.
Inyección de dependencias	Técnica por la cual un componente recibe sus colaboradores desde fuera en lugar de construirlos, facilitando la sustitución en pruebas.
JTI	*JWT ID*: identificador único de un token según RFC 7519; garantiza unicidad y habilita la revocación individual.
JWT	Token firmado y autocontenido que transporta la identidad y el rol del usuario entre cliente y servidor.
Migración	Script versionado que aplica un cambio concreto al esquema de la base de datos.
ORM	Capa que traduce entre objetos de Python y filas de una base de datos relacional.
Puerto	Interfaz abstracta que el dominio declara para expresar lo que necesita del exterior, sin comprometerse con una implementación.
Pydantic	Librería de validación y serialización de datos basada en anotaciones de tipo de Python.
RBAC	Control de acceso basado en roles: los permisos se asignan al rol y el rol al usuario.
Refresh token	Token de vida larga que permite obtener nuevos access tokens sin volver a pedir la contraseña.
Respx	Librería que intercepta peticiones HTTP salientes en pruebas para simular servicios externos de forma determinista.
Salt	Valor aleatorio que se combina con la contraseña antes de hashearla, de modo que contraseñas iguales produzcan hashes distintos.
SameSite	Directiva de cookie que controla si el navegador la envía en peticiones originadas desde otros sitios.
Swagger / OpenAPI	Especificación estándar de APIs REST y su interfaz interactiva, generada automáticamente por FastAPI.
UUID	Identificador universalmente único de 128 bits; evita exponer el número de registros y dificulta la enumeración.
Uvicorn	Servidor ASGI de alto rendimiento sobre el que se ejecuta la aplicación FastAPI.
XSS	Inyección de scripts maliciosos en una página web; se mitiga, entre otras medidas, con cookies `HttpOnly`.